Runbook: running and managing this repository

Generated 2026-10-13 from docs/RUNBOOK.md · 295 lines of source

Contents

Cheat sheet
1. Prerequisites
2. First-time setup
3. Running things locally
4. Everyday commands
5. Environment files
6. Database
7. Making changes
8. Tests and quality gates
9. Building
10. Keeping dependencies healthy
11. Releasing and operating
12. Troubleshooting
13. Rules that protect production

How to set it up, run it, change it safely and keep it healthy. For how the pieces fit together read ARCHITECTURE.md; for deploying read DEPLOYMENT.md.

Cheat sheet

```
corepack enable && pnpm install        # once, and after pulling dependency changes
pnpm db:local                          # embedded Postgres with the shared schema (own terminal)

pnpm --filter @wadzzo/ar     dev -p 3000     # AR collector
pnpm --filter @wadzzo/brand  dev -p 3001     # brand portal
pnpm --filter @wadzzo/market dev -p 3002     # fan marketplace
pnpm --filter @wadzzo/express dev            # task server on :4000

pnpm typecheck && pnpm lint && pnpm test && pnpm format:check    # the CI gate
pnpm env:check                                                    # env files agree
```

1. Prerequisites

Tool	Version	Notes
Node.js	`>=20.19` (`.nvmrc` says 20)	`nvm use` picks it up
pnpm	`12.9.1` (the `packageManager` field)	`corepack enable` installs the right one; do not use npm or yarn here
Git	any recent	
Postgres	optional	`pnpm db:local` gives you one with no install
Expo tooling	only for `apps/mobile`	it installs with **npm** on its own, see 8

2. First-time setup

```
git clone <repo> wadzzo-monorepo && cd wadzzo-monorepo
corepack enable
pnpm install                  # also runs `prisma generate` through @wadzzo/db
```

Then give each app you want to run its environment:

```
cp apps/brand/.env.example apps/brand/.env.local      # fill in; the file explains every variable
cp apps/market/.env.example apps/market/.env.local
cp apps/ar/.env.example    apps/ar/.env.local
cp apps/express-wadzzo/.env.example apps/express-wadzzo/.env
```

Real values are git-ignored (`.env` , `.env.*`); only `.env.example` is tracked. A teammate's real `.env.development` / `.env.production` files are shared out of band, never through git. See 5.

3. Running things locally

3.1 Database

Option	How	When
Embedded (`pnpm db:local`)	Postgres on `127.0.0.1:54329` , the schema from `packages/db` , and the two platform rows (`wadzzo` , `clintoncounty`). **In memory**: gone when you stop it.	quick checks, tests by hand, no network
A development database	`DATABASE_URL` in the app's `.env.local`	working with realistic data

For the embedded one use:

```
DATABASE_URL=postgresql://postgres:postgres@127.0.0.1:54329/postgres
DATABASE_POOL_MAX=1
```

It accepts **one connection at a time**, so run **one app at a time** against it.

3.2 The apps

Use fixed ports so the URLs in `.env.example` line up:

App	Command	Port	URL variables to match
AR (`web`)	`pnpm --filter @wadzzo/ar dev -p 3000`	3000	`NEXT_PUBLIC_URL` , `NEXTAUTH_URL` = `http://localhost:3000`
Brand (`brand`)	`pnpm --filter @wadzzo/brand dev -p 3001`	3001	`NEXT_PUBLIC_URL` , `NEXTAUTH_URL` = `http://localhost:3001` ; `NEXT_PUBLIC_WADZZO_AR_URL=http://localhost:3000`
Market (`app`)	`pnpm --filter @wadzzo/market dev -p 3002`	3002	`NEXT_PUBLIC_URL` , `NEXTAUTH_URL` = `http://localhost:3002` ; `NEXT_PUBLIC_BRAND_PORTAL_URL=http://localhost:3001`
Task server	`pnpm --filter @wadzzo/express dev`	4000	brand: `EXPRESS_SERVER_URL=http://localhost:4000` , same `NEXTAUTH_SECRET` on both

Notes:

- `pnpm dev` (everything at once) works, but the apps then take whatever ports are free. Prefer the explicit commands above.
- Each app's `predev` hook runs `sync:platform`, which writes `src/styles/platform-theme.css` and `public/platform/**` for the current `PLATFORM_SLUG`. They are generated and git-ignored.
- AR runs on webpack (`--webpack`, already in its script); brand and market use Turbopack.
- Outside production the brand app **requires** `EXPRESS_SERVER_URL` for the map agent and hotspot drops. It will not guess, so testing never reaches the production task server.

3.3 Another platform (Clinton County)

```
cd apps/brand
cp .env.clintoncounty .env.development.local     # overrides only; the rest comes from
.env.development
pnpm dev -p 3101
rm .env.development.local                         # back to Wadzzo
```

The theme, branding and assets follow `PLATFORM_SLUG`. Use a separate browser profile per platform: both share the `localhost` login cookie.

3.4 The mobile app

`apps/mobile` is **not** a workspace member and installs with npm (Metro wants a flat `node_modules`):

```
cd apps/mobile
npm install
npx expo start
```

It talks to the AR app's `/api/mobile/v1`. Copy `.env.example` to `.env` and set `EXPO_PUBLIC_API_URL` to your AR app (a phone needs your machine's LAN address, not `localhost`).

4. Everyday commands

Command	What it does
`pnpm typecheck`	`tsc` for every app and package (apps run `next typegen` first)
`pnpm lint`	ESLint; errors fail, warnings are a to-do list
`pnpm test`	all unit tests (Vitest, no network, no database)
`pnpm format` / `pnpm format:check`	Prettier for the whole repo
`pnpm db:generate`	regenerate the Prisma client after a schema edit
`pnpm db:studio`	Prisma Studio against `DATABASE_URL`
`pnpm env:check`	verifies the real env files (see 5)
`pnpm build`	every app and package (see 9 before running it locally)

Scope a command to one package (much faster):

```
pnpm --filter @wadzzo/api typecheck
pnpm --filter @wadzzo/api exec vitest run src/brand           # a folder
pnpm --filter @wadzzo/wallet exec vitest watch                 # watch mode
pnpm --filter @wadzzo/brand... typecheck                       # the app + its dependencies
```

Turbo caches `typecheck`, `lint` and `test`. After editing the Prisma schema run `pnpm db:generate` first or types will be stale.

5. Environment files

File (in `apps/<app>/`)	In git	Holds
`.env.example`	yes	every variable the app reads, grouped, with a description
`.env.local`	no	your machine's values (loaded in dev and prod mode)
`.env`, `.env.development`, `.env.production`	no	the shared real values (same sections as the example)
`.env.clintoncounty`, `.env.clintoncounty.production`	no	Clinton County overrides only (brand and AR)

Next.js loads, highest first: `.env.<mode>.local`, `.env.local`, `.env.<mode>`, `.env`. A variable already in the shell wins over all of them.

Check before you deploy or after any edit:

```
pnpm env:check            # both modes
pnpm env:check production
```

It never prints secrets (it compares hashes). It fails when something that must be identical across the apps differs (database, issuer key, platform wallets, bucket, Firebase), when the task server's `NEXTAUTH_SECRET` differs from brand's, or when a production URL doesn't match the app's domain.

Adding an environment variable

1. Read it through the right door: API code via `packages/api/src/env.ts` (add the name to `REQUIRED` or `OPTIONAL`; it is read on first use, never at import); app code via the app's `src/env.js`.
2. Add it to every affected `apps/<app>/.env.example` with a one-line description.
3. If the apps must agree on it, add it to `MUST_MATCH` in `scripts/check-env.mjs`.
4. Set it in the Vercel projects / on the task server (DEPLOYMENT.md).

6. Database

- The schema lives **only** in `packages/db/prisma/schema.prisma`; every app and the task server generate their client from it.
- Import models and enums from `@wadzzo/db` (browser-safe) and the client `db` from `@wadzzo/db/server`.
- **Never** run `prisma db push` by habit. It can drop data on a shared database, so the script refuses unless you say so:

  ```
  CONFIRM_DB_PUSH=yes DATABASE_URL="<the database you mean>" pnpm --filter @wadzzo/db push
  ```

Print `DATABASE_URL`'s host first. Production is changed from a trusted machine, additive changes first.

- Hand-written one-off SQL (new tables, backfills) lives in `packages/db/prisma/sql/`, named by date, applied in date order.
- Demo data: `packages/db/prisma/seed-clinton-demo.ts` and `seed-events.ts` (read their headers; they take `--clean`). AR has `pnpm --filter @wadzzo/ar seed:murals`.
- New rows of platform-scoped models **must set** `platformId`. A test (`packages/api/src/brand/__tests__/platform-stamping.test.ts`) scans every app's creates and fails if one is missing.

7. Making changes

Where code goes

You are adding	Put it in
A tRPC procedure or REST handler	`packages/api/src/<brand|market|ar>/routers/` (`rest/`, `mobile/`, `game/` for REST); apps only re-export
Logic two apps need	`packages/api/src/shared/` or the package that owns the concept
A form or request schema used by UI and API	`@wadzzo/schemas`
Pure AR types, distance, rarity, mural constants	`@wadzzo/ar-core` (no server imports, usable by the screens and mobile)
A UI component or page pattern	`@wadzzo/ui` (`components/` shadcn, `patterns/` shared layouts)
Wallet / signing / network config	`@wadzzo/wallet`
Branding, theme, assets for a platform	`@wadzzo/platform` (PLATFORMS.md)
A screen or page	the app (`apps/<app>/src/app` or `components`)

Rules (also in ARCHITECTURE.md): packages never import from an app or use `~/`; no `import "server-only"` in packages; settings come from `@wadzzo/api/env`; failures are `AppError` with an `ErrorCodes` value, never a bare `Error`; no duplicated helper between apps.

Recipes

A procedure

1. Create or extend a router under `packages/api/src/<app>/routers/`; build it with `createTRPCRouter`, `publicProcedure`, `protectedProcedure` from `@wadzzo/api` (brand also has `adminProcedure`, `creatorProcedure` in `brand/trpc.ts`).
2. Validate input with zod; throw `new AppError(ErrorCodes.X, { message })`.
3. Mount it in that app's root router (`<app>/router.ts` or the domain's `root.ts`).
4. A procedure that takes `signWith` is checked against the session automatically; do not add your own check.
5. Add a test next to it (`__tests__/`) using `createCallerFactory` with a stub `db`.

An error code: add it to `packages/errors/src/error-codes.ts` with its HTTP status, tRPC code and a safe default message.

A REST route: put the handler in `packages/api/src/<app>/{rest,mobile,game}/...`; the app's route file is one re-export (`export { GET } from "@wadzzo/api/ar/rest/..."`). Route-segment config (`dynamic`, `config.api.bodyParser`) must stay literal in the app's route file.

A schema change: edit `schema.prisma` → `pnpm db:generate` → typecheck → write a dated SQL file for production if data must move → apply as in 6.

A dependency: add the version to the `catalog:` in `pnpm-workspace.yaml`, then `"dep": "catalog:"` in the package that uses it. Do not put a version number in a package's own `package.json`. Packages that use it declare it themselves (nothing is hoisted for you).

A new package: copy `packages/geo` (`package.json`, `tsconfig.json`, `eslint.config.mjs`, `vitest.config.mts`), set `name`, `exports` and scripts, add `"@wadzzo/<name>": "workspace:*"` to the consumers, `pnpm install`.

A shadcn component: `cd packages/ui && pnpm dlx shadcn@latest add <name>`, then change the generated imports to the package's relative ones (`../lib/cn`) and run `pnpm format`.

8. Tests and quality gates

Layer	Where	Runs in CI	Needs
Unit	every package's `src/__tests__`, apps' `tests/`	yes	nothing (`SKIP_ENV_VALIDATION=1` is set)
End-to-end	`apps/{brand,market,ar}/e2e`, `pnpm test:e2e`	no	a dev database and Stellar testnet

The CI gate (`.github/workflows/ci.yml`) is: format check, typecheck, lint, unit tests. Run all four locally before a PR. Lint warnings are allowed; **new** warnings in code you touched are not welcome.

Commit and PR rules:

- Never commit `.env*` files other than `.env.example`.
- Do not add a `Co-Authored-By: Claude` trailer or any other AI attribution.
- Keep schema changes, dependency bumps and refactors in their own commits.

9. Building

```
pnpm --filter @wadzzo/brand build
```

Read this first:

- A build needs the `NEXT_PUBLIC_*` values and (unless you pass `SKIP_ENV_VALIDATION=1`) the whole validated environment. For a smoke build with no real env: `SKIP_ENV_VALIDATION=1 NEXT_PUBLIC_STELLAR_NETWORK=testnet pnpm --filter @wadzzo/ar build`. Market additionally needs `NEXT_PUBLIC_URL` and `NEXT_PUBLIC_BRAND_PORTAL_URL`.
- If `apps/<app>/.env.production` exists on your machine it **is loaded** by `next build`. Pin `DATABASE_URL` to something harmless (`postgresql://postgres:postgres@127.0.0.1:1/none`) so a build can never touch a real database.
- The market build prerenders `/sitemap.xml`, which queries the database. Without a reachable database it stops there after 18 of 19 pages; that is expected.
- `next build` and `next dev` in the same app share `.next`. Stop the dev server of that app first, or build in a separate checkout.

The task server builds with `pnpm --filter @wadzzo/express build` (tsup) and starts with `node dist/index.js`.

10. Keeping dependencies healthy

```
pnpm outdated -r            # what is behind
```

Bump the version in the `catalog:` of `pnpm-workspace.yaml`, `pnpm install`, then run the CI gate and the e2e tests of the apps that use it. pnpm may add `minimumReleaseAgeExclude` entries for very new releases; that is intended.

Held back on purpose:

Package	Why
`prisma` / `@prisma/client` 7.10	8 is a release candidate
`typescript` 6.0.x	`typescript-eslint` does not support 7 yet
the web3 stack (`@web3modal/*`, `@walletconnect/*`, Albedo, xBull, HOT)	pinned to what the wallets were verified with

TypeScript 6 does not include `@types/*` implicitly: a package that uses Node globals sets `"types": ["node"]` and lists `@types/node`.

11. Releasing and operating

- Branches: `main` (production), `develop` (development environment). Web apps deploy from Vercel on push; the task server from `.github/workflows/deploy-express.yml`. Details, rollbacks and the checks after a deploy are in DEPLOYMENT.md.
- Before a release: CI green, `pnpm env:check production`, and for schema changes the SQL applied first.
- Secrets rotation: a value used by several apps (`ISSUER_ENCRYPTION_KEY`, `MOTHER_SECRET`, `NEXTAUTH_SECRET`, the database password) changes in **every** project and the task server in one window, then `pnpm env:check`.

12. Troubleshooting

Symptom	Cause and fix
`Cannot find module '.prisma/client'` or stale model types	`pnpm db:generate`
`CONFIG_MISSING` naming a variable	That setting isn't set for this app; compare with its `.env.example` (API settings are read on first use, so it appears when a feature is used)
`Invalid environment variables` at start	The app's `src/env.js` rejected a value; the message names the variable
`This module cannot be imported from a Client Component module`	an `import "server-only"` reached a pages-router route; remove it from package code
"The task server isn't configured for this environment"	`EXPRESS_SERVER_URL` is unset outside production (by design); set it
Port already in use	something is listening there; find it with `lsof -nP -iTCP:3001 -sTCP:LISTEN` and check before killing: it may be a teammate's or your own other terminal
Prisma connection errors with `pnpm db:local`	it accepts one connection: one app at a time, `DATABASE_POOL_MAX=1`
`tsc` says `Cannot find name 'process'` in a package	add `@types/node` and `"types": ["node"]` to that package (TypeScript 6)

Symptom	Cause and fix
Theme or logo looks like the wrong platform	`PLATFORM_SLUG` differs from the one the app last synced; restart `dev` so `sync:platform` reruns
Login redirects in a loop locally	`NEXTAUTH_URL` doesn't match the port you're on, or two platforms share the `localhost` cookie
A package change doesn't show up in an app	restart `next dev`; if the package is new and TS-only add it to `transpilePackages` in the app's `next.config.js`
`pnpm install` refuses a lockfile	someone edited a `package.json` without installing; run `pnpm install` and commit `pnpm-lock.yaml`
In zsh, commands vanish after a loop	don't use `path` as a loop variable in zsh; it is tied to `PATH`

13. Rules that protect production

1. **Know which database you are pointed at** before `db push`, seeds or e2e tests: print the host.
2. **Never put real secrets in a tracked file**, a commit message or a screenshot.
3. **Do not run the e2e tests, seeds or `db push` against production**. They are for development databases.
4. **Do not change `/api/mobile/v1` contracts** (routes, parameters, response shapes, the `{ error: { code, message } }` body). Add routes instead; the shipped mobile app depends on them.
5. **One network per deployment**: mainnet and testnet are never mixed in one environment.
6. **The task server runs as one instance.** Deploy it when its queue is idle (`GET /health`).
7. **Do not kill processes you didn't start.** A busy port is often someone's dev server.